

---

# INTRODUCTION AU MACHINE LEARNING

---

NDIAYE Maïrame

Master 2 Méthodes Informatiques Appliquées à la Gestion des Entreprises

Université Paris Dauphine – PSL

2026-2027

# Préambule

Le *Machine Learning* (apprentissage automatique) est aujourd'hui au cœur de la science des données et de l'intelligence artificielle. Que l'on parle de transformation numérique des entreprises, de *Big Data*, d'intelligence artificielle générative ou de souveraineté technologique, il est devenu incontournable. Ses applications sont nombreuses et variées : moteurs de recherche, systèmes de recommandation, médecine, analyse des réseaux sociaux, vision par ordinateur, traduction automatique, détection de fraude ou encore modèles de langage.

Mais le Machine Learning est aussi un domaine en perpétuel mouvement. Les méthodes évoluent, les modèles gagnent en complexité et de nouvelles idées apparaissent à un rythme tel qu'une partie des outils que nous utiliserons demain n'existe probablement pas encore aujourd'hui. Dans un tel contexte, apprendre ne peut pas simplement consister à mémoriser un ensemble d'algorithmes.

C'est dans cet esprit que ce cours a été conçu. Son ambition n'est pas de dresser une carte exhaustive du Machine Learning, car une telle carte serait inévitablement incomplète et rapidement dépassée. Il cherche plutôt à fournir une **boussole** : un ensemble de connaissances, d'intuitions et de principes suffisamment solides pour permettre de s'orienter dans un paysage qui continuera de se transformer.

Nous étudierons ainsi les principaux algorithmes du Machine Learning, mais toujours en cherchant à comprendre les questions qui les ont fait naître. Quel problème cherche-t-on à résoudre ? Pourquoi cette méthode fonctionne-t-elle ? Qu'apprend réellement le modèle ? Quelles hypothèses faisons-nous ? Pourquoi une nouvelle méthode a-t-elle été nécessaire ? Quelles en sont les limites ? Les mathématiques accompagneront cette exploration lorsqu'elles permettront de préciser une intuition et de révéler les mécanismes qui se cachent derrière les algorithmes.

Au fond, l'objectif de ce cours dépasse donc les méthodes qui y sont présentées. Il s'agit d'apprendre à regarder une idée nouvelle sans qu'elle soit entièrement étrangère, à reconnaître derrière elle des principes déjà rencontrés et, surtout, à savoir quelles questions poser avant même d'en connaître les réponses.

Car dans un domaine appelé à se transformer tout au long de nos vies, le savoir le plus durable n'est peut-être pas celui qui nous permet de connaître le monde tel qu'il est aujourd'hui, mais celui qui nous prépare à comprendre celui qui viendra demain.

# Table des matières

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Préambule</b>                                                        | <b>1</b>  |
| <b>1 Introduction au Machine Learning</b>                               | <b>1</b>  |
| 1.1 Machine Learning, qu'est-ce que c'est ? . . . . .                   | 1         |
| 1.2 Différentes approches en Machine Learning . . . . .                 | 2         |
| 1.3 Concept fondamental de généralisation . . . . .                     | 5         |
| 1.4 La sélection de modèles . . . . .                                   | 6         |
| 1.4.1 Train, Validation, Test . . . . .                                 | 7         |
| 1.4.2 Validation croisée . . . . .                                      | 8         |
| <b>2 Régression linéaire et fondements de l'apprentissage supervisé</b> | <b>10</b> |
| 2.1 L'apprentissage supervisé : formalisation . . . . .                 | 10        |
| 2.1.1 Fonction de perte . . . . .                                       | 12        |
| 2.1.2 Fonction de coût . . . . .                                        | 12        |
| 2.1.3 Descente de gradient . . . . .                                    | 13        |
| 2.2 Régression Linéaire Classique . . . . .                             | 14        |
| 2.3 Évaluer la performance d'une régression . . . . .                   | 17        |
| 2.4 Complexité des modèles . . . . .                                    | 19        |
| 2.4.1 Régressions polynomiales . . . . .                                | 19        |
| 2.4.2 Compromis Biais-Variance . . . . .                                | 21        |
| 2.4.3 Régressions pénalisées . . . . .                                  | 23        |

# Chapitre 1

## Introduction au Machine Learning

Dans ce chapitre, nous proposons d'introduire le Machine Learning et les différentes familles de problèmes qu'il permet de résoudre.

### 1.1 Machine Learning, qu'est-ce que c'est ?

Les termes *Intelligence Artificielle* (IA) et *Machine Learning* (ML) sont souvent confondus, bien qu'ils ne désignent pas exactement la même chose.

L'Intelligence Artificielle désigne l'ensemble des méthodes visant à concevoir des systèmes capables de réaliser des tâches associées à l'intelligence humaine : raisonner, planifier, comprendre le langage, percevoir des images ou des sons, prendre des décisions, jouer à des jeux, etc.

Le Machine Learning constitue une sous-branche de l'IA dont l'objectif est de permettre à une machine d'apprendre automatiquement à partir de **données**, sans que toutes les règles nécessaires à la résolution du problème aient été explicitement programmées.

L'un des premiers exemples historiques de Machine Learning est le programme de jeu de dames développé par Arthur Samuel à la fin des années 1950. Contrairement aux programmes classiques de l'époque, ce système n'était pas basé sur un ensemble fixe de stratégies codées à la main.

Son approche consistait à faire jouer deux programmes l'un contre l'autre de manière répétée. Initialement, leurs stratégies étaient essentiellement aléatoires, puis les programmes étaient récompensés ou pénalisés selon l'issue des parties. Progressivement, ils apprenaient quelles situations augmentaient leurs chances de victoire et quelles décisions menaient à des défaites. Autrement dit, le programme améliorait ses performances par **l'expérience**.

Aujourd'hui, le Machine Learning occupe une place centrale dans de nombreux domaines scientifiques, industriels et économiques. Ses applications sont extrêmement variées : moteurs de recherche, systèmes de recommandation, traitement automatique du langage, détection de fraude, analyse de réseaux sociaux, vision par ordinateur, médecine, finance, recherche génomique, véhicules autonomes, etc.

À l'intersection des statistiques, des mathématiques appliquées et de l'informatique, le Machine Learning repose essentiellement sur deux piliers fondamentaux :

- **les données**, qui sont les exemples à partir desquels le système va apprendre ;
- **l'algorithme d'apprentissage**, qui est la procédure utilisée pour construire un modèle à partir de ces données.

Ces deux piliers sont indissociables. D'une part, même le meilleur algorithme d'apprentissage ne pourra pas produire un modèle performant à partir de données non pertinentes, incomplètes ou bruitées. Cette idée est souvent résumée par l'expression *garbage in, garbage out* : des données de mauvaise qualité conduisent inévitablement à des prédictions de mauvaise qualité. D'autre part, disposer de bonnes données ne garantit pas non plus de bons résultats si l'algorithme choisi n'est pas adapté au problème considéré.

Dans ce cours, nous nous concentrerons principalement sur les algorithmes d'apprentissage et les principes mathématiques qui les sous-tendent. Cependant, il est important de garder à l'esprit qu'une grande partie du travail réel en Machine Learning concerne également l'ingénierie des données : collecte, nettoyage, transformation, représentation des variables, analyse exploratoire ou encore sélection des caractéristiques pertinentes.

## 1.2 Différentes approches en Machine Learning

La grande diversité des problèmes traités en Machine Learning soulève naturellement la question suivante : *existe-t-il un algorithme d'apprentissage universellement efficace ?*

Le théorème *No-Free-Lunch* apporte une réponse fondamentale à cette question : il n'existe pas d'algorithme d'apprentissage universel performant sur l'ensemble des tâches possibles. Un modèle très efficace dans un certain contexte pourra ainsi se révéler médiocre dans un autre.

Cette impossibilité provient notamment de la grande diversité des distributions de probabilités susceptibles de générer les données observées. En effet, tout algorithme d'apprentissage repose, explicitement ou implicitement, sur certaines hypothèses concernant la structure des données. Lorsque ces hypothèses sont adaptées au problème étudié, l'algorithme peut obtenir de très bonnes performances. À l'inverse, lorsque ces hypothèses sont inadaptées, ses performances peuvent fortement se dégrader.

Le choix d'un algorithme dépend donc toujours de la nature des données et du problème considéré.

Face à cette diversité de situations, plusieurs grandes approches du Machine Learning ont été développées. Bien qu'elles reposent toutes sur l'exploitation des données afin d'apprendre automatiquement un modèle, elles partagent également un principe fondamental commun : l'apprentissage consiste toujours à optimiser un certain objectif. Selon les approches, cet objectif peut prendre des formes différentes : minimiser une erreur de prédiction, maximiser la compacité de groupes de données, ou encore maximiser une récompense obtenue au cours d'interactions avec un environnement.

## Apprentissage supervisé

L'apprentissage supervisé est probablement l'approche la plus répandue en Machine Learning. Son objectif est d'apprendre à réaliser des prédictions à partir d'exemples étiquetés, c'est-à-dire de données pour lesquelles la réponse attendue est connue.

Le terme *supervisé* fait référence au fait que l'apprentissage est guidé par ces étiquettes, de manière analogue à un enseignant corrigeant progressivement les réponses de ses élèves à partir d'exemples. L'algorithme cherche alors à apprendre une relation entre les données d'entrée et la sortie attendue afin de pouvoir généraliser à de nouvelles observations jamais vues auparavant.

Selon la nature des étiquettes à prédire, on distingue principalement deux grandes catégories de problèmes.

Lorsque l'espace des étiquettes est *discret* et *fini*, on parle de tâche de **classification**. L'objectif consiste alors à attribuer une catégorie à chaque observation. Voici quelques exemples de problèmes de classification :

- déterminer si un email est un spam ou non ;
- déterminer si une image contient un chat ;
- détecter si une transaction bancaire est frauduleuse ;
- identifier la langue dans laquelle un texte est écrit ;
- reconnaître l'espèce à laquelle appartient une plante.

Lorsque l'espace des étiquettes est *continu* (généralement réel), on parle de tâche de **régression**. L'objectif consiste alors à prédire une quantité numérique. Voici quelques exemples de problèmes de régression :

- prédire le prix d'un bien immobilier ;
- estimer le rendement d'un champ de maïs ;
- prédire le nombre de clics sur une publicité ;
- estimer l'affinité de liaison entre deux molécules ;
- prédire la consommation électrique d'un bâtiment.

## Apprentissage non supervisé

Dans le cadre de l'apprentissage non supervisé, les données ne possèdent pas d'étiquettes associées. Aucun signal explicite n'indique à l'algorithme ce qu'il doit apprendre. Le système cherche alors à mettre en évidence des régularités, des groupes ou des représentations pertinentes à partir des seules données observées.

Lorsque l'objectif consiste à identifier des groupes d'observations similaires, on parle de tâche de **clustering** ou de **partitionnement**. L'idée générale est de regrouper ensemble les données qui se ressemblent selon une certaine mesure de similarité. Voici quelques exemples de problèmes de clustering :

- segmenter des clients ayant des comportements d'achat similaires ;

- regrouper des documents traitant de sujets proches ;
- identifier des groupes de patients présentant des symptômes similaires ;
- compresser une image en regroupant des pixels de couleurs proches.

Il est important de noter qu'un algorithme de clustering produira toujours des groupes, même lorsque ceux-ci n'ont pas nécessairement de signification pertinente dans le contexte étudié. L'interprétation des résultats dépend donc fortement du problème métier considéré.

Lorsque l'objectif consiste à représenter les données dans un espace de plus faible dimension tout en conservant le maximum d'informations possible, on parle de tâche de **réduction de dimension**. La réduction de dimension permet notamment de :

- réduire les coûts de stockage et de calcul ;
- visualiser plus facilement des données complexes ;
- limiter certains effets liés à la grande dimension ;
- parfois d'améliorer les performances d'algorithmes d'apprentissage supervisé.

## Apprentissage semi-supervisé

L'apprentissage semi-supervisé se situe à mi-chemin entre l'apprentissage supervisé et non supervisé. Dans ce cadre, seule une petite partie des données possède des étiquettes, tandis que le reste des observations est non annoté.

Cette approche est particulièrement utile lorsque les données sont faciles à collecter mais coûteuses à annoter manuellement. Par exemple, il est relativement simple d'accumuler des millions d'images sur Internet, mais associer une étiquette correcte à chacune d'entre elles peut nécessiter un travail humain considérable.

L'objectif est alors d'exploiter simultanément les quelques données étiquetées et la structure des données non étiquetées afin d'améliorer les performances d'apprentissage.

L'apprentissage semi-supervisé constitue un domaine de recherche actif mais dépasse le cadre de ce cours.

## Apprentissage par renforcement

Dans l'apprentissage par renforcement, un agent interagit avec un environnement en réalisant des actions. En retour, il reçoit des récompenses positives ou négatives selon la qualité de ses décisions.

L'objectif n'est plus de prédire directement une étiquette, mais d'apprendre une stratégie d'action, appelée *politique*, maximisant la récompense cumulée au cours du temps.

Une difficulté importante provient du fait que les conséquences d'une action peuvent apparaître beaucoup plus tard. Par exemple, dans des jeux comme les échecs ou le go, une mauvaise décision peut n'avoir des effets visibles qu'après plusieurs dizaines de coups.

L'apprentissage par renforcement est aujourd'hui utilisé dans des domaines tels que :

- les jeux ;
- la robotique ;
- les systèmes autonomes ;
- l'optimisation de stratégies de décision.

Ce domaine dépasse largement le cadre de ce cours et ne sera abordé qu'à titre introductif.

En pratique, l'apprentissage supervisé et l'apprentissage non supervisé représentent la grande majorité des applications industrielles du Machine Learning. Ce sont donc principalement ces deux grandes familles de méthodes que nous étudierons dans ce cours.

### 1.3 Concept fondamental de généralisation

Comme nous l'avons vu avec le théorème *No-Free-Lunch*, il n'existe pas de modèle universellement optimal. La qualité d'un algorithme dépend toujours du problème étudié, des données disponibles et des hypothèses faites sur leur structure.

Une question fondamentale se pose alors : *comment déterminer si un modèle est réellement performant ?*

Intuitivement, on pourrait penser qu'un bon modèle est un modèle capable de mémoriser parfaitement les exemples observés. Cependant, un tel modèle serait performant uniquement sur les données qu'il a observées et serait incapable de faire des prédictions correctes sur de nouvelles observations. Ce n'est pas vraiment ce que l'on a en tête lorsqu'on parle d'*apprentissage*.

L'objectif du Machine Learning n'est donc pas uniquement d'apprendre les données d'entraînement, mais surtout de construire des modèles capables de *généraliser* à des observations jamais vues auparavant.

**Definition 1** (Généralisation). *On appelle généralisation la capacité d'un modèle à faire des prédictions correctes sur des nouvelles données, qui n'ont pas été utilisées pour le construire.*

Ce concept est central en apprentissage automatique, car il conditionne directement la manière dont les modèles sont évalués et sélectionnés.

Dans ce cadre, deux phénomènes extrêmes peuvent apparaître lors de l'apprentissage.

Pour les illustrer, considérons le cas d'un étudiant préparant un examen à partir d'une série d'exercices corrigés.

- Si l'étudiant ne parvient pas à identifier les principes généraux du cours et applique des méthodes trop simplistes ou inadéquates, il échouera aussi bien sur les exercices d'entraînement que sur de nouveaux exercices. On parle alors de **sous-apprentissage**

(*underfitting*). Dans ce cas, le modèle possède une capacité insuffisante pour représenter la structure des données et présente des performances faibles à la fois sur les données d'entraînement et sur les données de test.

- À l'inverse, un étudiant peut apprendre par cœur les solutions des exercices sans en comprendre les concepts sous-jacents. Il obtiendra alors d'excellentes performances sur les exercices déjà rencontrés, mais sera incapable de résoudre des exercices légèrement différents. On parle alors de **surapprentissage** (*overfitting*). Dans ce cas, le modèle s'adapte excessivement aux données d'entraînement, au point d'apprendre également leurs fluctuations spécifiques ou leur bruit, qui ne sont pas représentatifs du problème général.

Ces deux situations illustrent un compromis fondamental en apprentissage automatique : un modèle doit être suffisamment flexible pour capturer la structure des données, tout en restant suffisamment contraint pour éviter d'apprendre les variations non généralisables de l'échantillon d'apprentissage.

Ce compromis est au cœur des méthodes de sélection de modèles, que nous introduisons dans la section suivante.

## 1.4 La sélection de modèles

Nous avons vu que l'objectif fondamental d'un modèle de Machine Learning n'est pas uniquement d'obtenir de bonnes performances sur les données d'entraînement, mais de généraliser correctement à de nouvelles données jamais observées.

Cette problématique soulève une question essentielle : *comment évaluer objectivement la capacité de généralisation d'un modèle et comparer plusieurs modèles entre eux ?*

À ce stade, il est important de distinguer deux notions proches mais conceptuellement différentes :

- **l'évaluation d'un modèle**, qui consiste à estimer sa capacité de généralisation sur des nouvelles données ;
- **la sélection de modèles**, qui consiste à comparer plusieurs modèles afin de choisir celui qui semble le plus adapté au problème considéré.

La première notion sera abordée progressivement tout au long du cours, en fonction des modèles étudiés. Cette section se concentre sur la seconde, en présentant le cadre expérimental classiquement utilisé en Machine Learning pour comparer et sélectionner des modèles.

Pour illustrer cette notion, considérons le cas d'un *hackathon* de Machine Learning. Dans ce type de compétition, plusieurs équipes cherchent à construire le modèle le plus performant sur une tâche donnée.

Par exemple, supposons qu'un organisateur fournisse un ensemble de données contenant différentes variables permettant de prédire le prix de l'essence. Les participants disposent alors d'un jeu de données d'apprentissage, composé des variables explicatives et des valeurs cibles associées. En revanche, les valeurs du jeu de données d'évaluation ne sont pas accessibles.

Chaque équipe doit donc entraîner un modèle sur les données d'apprentissage, produire des prédictions sur les nouvelles observations, puis soumettre ses résultats. Les performances sont ensuite évaluées en comparant les prédictions aux valeurs réelles, connues uniquement des organisateurs.

Ce cadre illustre une idée fondamentale du Machine Learning : un modèle doit être évalué sur des données qu'il n'a jamais observées lors de son apprentissage.

En pratique, les participants testent généralement plusieurs modèles, ajustent leurs paramètres et comparent leurs performances afin de sélectionner la meilleure approche possible. Cela soulève immédiatement une difficulté essentielle : *comment comparer correctement plusieurs modèles sans biaiser l'évaluation finale ?*

Pour répondre à cette question, il est nécessaire de distinguer plusieurs ensembles de données jouant des rôles spécifiques.

### 1.4.1 Train, Validation, Test

Soit  $\mathcal{D}$  un ensemble de données. Une approche classique en Machine Learning consiste à partitionner cet ensemble en trois sous-ensembles distincts :

1. **Train** : pour estimer les paramètres optimaux du modèle à partir des données d'apprentissage ;
2. **Validation** : pour évaluer et comparer différentes configurations du modèle (choix d'algorithme, hyperparamètres), sans intervenir dans l'estimation des paramètres ;
3. **Test** : pour obtenir une estimation finale et non biaisée de la capacité de généralisation du modèle sélectionné.

Cette séparation permet de distinguer clairement l'apprentissage, la sélection et l'évaluation finale, pour éviter toute contamination entre ces différentes étapes.

On comprends ainsi que lors d'un hackaton, les équipes scindent leur dataset avec les réponses en deux parties : *train* et *validation*. Cela est réalisée aléatoirement dans la majorité des cas, mais il faut s'assurer que cela n'introduise pas de biais. Par exemple, dans le cadre de la prédiction du prix de l'essence, une séparation temporelle inadéquate (entraîner sur des données anciennes et valider sur des données plus récentes sans en tenir compte) peut conduire à des estimations trompeuses des performances.

L'inconvénient majeur de cette approche est qu'elle fournit une estimation ponctuelle des performances du modèle. En effet, la séparation d'un jeu de données en ensembles d'entraînement, de validation et de test repose sur un découpage nécessairement arbitraire. Par conséquent, il est possible que, par simple effet du hasard, les sous-ensembles obtenus ne soient pas parfaitement représentatifs de la distribution globale des données, ce qui peut conduire à une estimation instable des performances.

Le cadre standard permettant de limiter ce problème est celui de la **validation croisée**.

### 1.4.2 Validation croisée

La validation croisée (*cross-validation*) est une méthode standard permettant d'obtenir une estimation plus robuste des performances d'un modèle, en réduisant la dépendance au découpage initial des données.

L'idée générale consiste à répéter plusieurs fois le processus de séparation entre ensemble d'entraînement et ensemble de validation, afin d'obtenir plusieurs estimations des performances, puis à les agréger afin de lisser l'effet des fluctuations aléatoires.

La méthode la plus couramment utilisée est la **K-Fold Cross-Validation**.

Le jeu d'entraînement est partitionné aléatoirement en  $K$  sous-ensembles de taille approximativement égale, appelés *folds*.

Le processus est le suivant :

- à chaque itération  $i \in \{1, \dots, K\}$ , un fold est utilisé comme ensemble de validation ;
- les  $K - 1$  autres folds sont utilisés comme ensemble d'entraînement ;
- le modèle est entraîné sur ces  $K - 1$  folds puis évalué sur le fold laissé de côté.

On obtient ainsi  $K$  estimations de la performance du modèle, chacune correspondant à un découpage différent des données.

La performance finale du modèle est généralement obtenue en moyennant les performances sur les  $K$  itérations :

$$Performance_{CV} = \frac{1}{K} \sum_{i=1}^K Performance_i$$

Cette moyenne permet d'obtenir une estimation plus stable et plus fiable de la capacité de généralisation du modèle. En ce sens, elle fournit une approximation plus robuste de la performance attendue du modèle sur des données non vues.

On parle de **Leave-One-Out Cross-Validation** lorsque  $K = n$ , où  $n$  désigne le nombre d'observations du jeu de données. Dans ce cas limite, chaque itération utilise  $n - 1$  observations pour l'apprentissage et une seule observation pour la validation. L'intuition derrière cette méthode est qu'un algorithme d'apprentissage est d'autant plus performant qu'il dispose d'un maximum de données pour son entraînement, ce qui conduit ici à des ensembles d'apprentissage presque complets à chaque itération.

Cette approche présente toutefois deux inconvénients majeurs. Le premier est son coût computationnel : elle nécessite l'entraînement de  $n$  modèles distincts, chacun sur  $n - 1$  observations, ce qui peut devenir onéreux lorsque  $n$  est grand. Le second est lié à la variance de l'estimation. Les modèles entraînés sont très proches les uns des autres, car leurs ensembles d'entraînement ne diffèrent que par une seule observation. En revanche, les évaluations reposent sur des ensembles de test composés d'observations distinctes, ce qui peut induire une variabilité importante des performances et rendre leur interprétation moins stable.

Une question naturelle consiste alors à se demander : *existe-t-il une valeur optimale de  $K$  permettant d'obtenir une estimation non biaisée et de variance minimale de la performance de généralisation ?*

Bengio et Grandvalet [1] montrent qu'il n'existe pas de valeur universelle optimale de  $K$ . Les travaux de synthèse d'Arlot et Celisse [2] montrent également que le choix de  $K$  résulte d'un compromis entre biais, variance et coût de calcul.

En pratique, il n'y a pas de règle pré-définies dans le choix de la valeur de  $K$ , c'est au jugement du data-scientist. Le choix final dépend généralement de la taille du jeu de données et des ressources disponibles.

Dans ce chapitre introductif, nous avons établi le cadre fondamental du Machine Learning : apprendre des données pour généraliser à des situations inédites. Le théorème *No-Free-Lunch* rappelle qu'aucune méthode n'est universellement optimale, ce qui justifie la diversité des approches et rend centrale la question de la sélection des modèles. Ces concepts formeront désormais la grille de lecture de l'ensemble des méthodes étudiées dans la suite du cours.

## Chapitre 2

# Régression linéaire et fondements de l'apprentissage supervisé

La régression linéaire est l'un des algorithmes d'apprentissage les plus utilisés au monde, en raison de sa simplicité et de ses bonnes propriétés analytiques. Elle appartient à la famille des algorithmes d'apprentissage supervisé.

Dans ce chapitre, nous verrons comment formaliser concrètement un problème d'apprentissage supervisé à travers cet exemple, puis comment construire un modèle de régression linéaire et ses principales variantes. Nous introduirons également les outils permettant d'évaluer les performances d'un modèle de régression, afin de mesurer sa capacité de généralisation.

### 2.1 L'apprentissage supervisé : formalisation

La régression linéaire n'est qu'un cas particulier d'un problème plus général : celui de l'apprentissage supervisé, dans lequel on cherche à apprendre une fonction de prédiction à partir d'exemples étiquetés.

Pour chaque problème de Machine Learning supervisé on dispose d'un dataset  $\mathcal{D}$  formé de la manière suivante :

$$\mathcal{D} = \left\{ (x^{(i)}, y^{(i)}) \mid \forall i \leq \text{Nombre d'observations } n, x^{(i)} \in \mathbb{R}^{\text{Nombre d'informations } d}, y^{(i)} \in \mathcal{Y} \right\} \quad (2.1)$$

L'ensemble  $\mathcal{Y}$  désigne l'espace des étiquettes. Dans le cas d'une régression, on a généralement  $\mathcal{Y} \subseteq \mathbb{R}$ . Dans le cas d'une classification,  $\mathcal{Y}$  est un ensemble fini de classes.

Un dataset  $\mathcal{D}$  est donc l'ensemble des paires  $(x^{(i)}, y^{(i)})$  où  $x^{(i)}$  est un vecteur de  $d$  variables explicatives et  $y^{(i)} \in \mathcal{Y}$  est un nombre ou une classe d'intérêt associée à l'observation  $i$ . On définit les notations :

—  $X$  : la matrice des informations :

$$X = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_d^{(n)} \end{pmatrix}$$

—  $y$  : le vecteur des réponses :

$$y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{pmatrix}$$

L'objectif de l'apprentissage supervisé est d'utiliser les exemples contenus dans  $\mathcal{D}$  afin d'apprendre une fonction de prédiction permettant d'estimer la valeur de  $y$  à partir des caractéristiques  $x$ .

Même lorsqu'une relation existe entre les variables explicatives et la variable à prédire, les observations sont rarement parfaitement déterministes. Certaines informations pertinentes peuvent être absentes des données, les mesures peuvent être bruitées et certains phénomènes comportent une part d'aléa intrinsèque.

On suppose ainsi qu'il existe une fonction inconnue

$$f : \mathbb{R}^d \rightarrow \mathcal{Y}, \forall i \leq n, y^{(i)} = f(x^{(i)}) + \epsilon^{(i)}$$

où  $\epsilon^{(i)}$  représente un terme de bruit regroupant l'ensemble des facteurs non observés ou imprévisibles.

L'objectif n'est pas seulement d'obtenir de bonnes prédictions sur les observations présentes dans  $\mathcal{D}$ , mais de construire une fonction capable de produire de bonnes prédictions sur de nouvelles observations issues du même phénomène. Autrement dit, on cherche à construire une fonction qui généralise correctement.

Dans la pratique, la fonction  $f$  est inconnue et ne peut pas être observée directement. Nous allons donc considérer une famille de fonctions candidates notées  $f_\theta$ , paramétrées par un vecteur de paramètres  $\theta \in \mathbb{R}^{d'}$ . L'apprentissage consiste alors à déterminer les valeurs des paramètres  $\theta$  qui permettent à la fonction  $f_\theta$  d'approcher au mieux la relation recherchée entre les variables explicatives et la variable cible.

Il reste alors une question fondamentale : *comment mesurer la qualité d'une fonction de prédiction et comparer plusieurs modèles entre eux ?*

Pour répondre à cette question, il est nécessaire de définir une mesure quantitative de l'erreur commise par le modèle. C'est le rôle des fonctions de perte et des fonctions de coût.

### 2.1.1 Fonction de perte

Pour déterminer quels paramètres  $\theta$  sont les plus adaptés à notre problème, il est nécessaire de quantifier la qualité des prédictions produites par le modèle. Nous devons donc définir une mesure de l'erreur commise lorsque le modèle prédit une valeur différente de la valeur observée.

La première notion fondamentale est celle de fonction de perte.

**Definition 2** (Fonction de perte). *Une fonction de perte est une fonction*

$$\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+$$

*qui associe à une valeur réelle  $y$  et à une prédiction  $\hat{y}$  une quantité représentant l'erreur commise par le modèle.*

Plus la valeur de la fonction de perte est élevée, plus la prédiction est éloignée de la valeur attendue.

### 2.1.2 Fonction de coût

La fonction de perte permet de mesurer l'erreur commise sur une observation individuelle. Cependant, l'apprentissage doit tenir compte de l'ensemble des observations du dataset.

Il est donc nécessaire d'agréger les pertes individuelles afin d'obtenir une mesure globale de la qualité du modèle. C'est le rôle de la fonction de coût.

**Definition 3** (Fonction de coût). *La fonction de coût mesure l'erreur globale du modèle sur l'ensemble des données d'apprentissage.*

*Elle est généralement définie comme la moyenne des pertes individuelles :*

$$\mathcal{C}(f_\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y^{(i)}, f_\theta(x^{(i)}))$$

La fonction de coût associe donc à chaque choix des paramètres  $\theta$  une valeur numérique représentant la qualité globale du modèle sur les données d'apprentissage.

L'objectif de l'apprentissage consiste alors à trouver les paramètres qui minimisent cette quantité :

$$f_{\theta^*} = \arg \min_{f_\theta} \mathcal{C}(f_\theta) \quad (2.2)$$

Ainsi, un problème d'apprentissage supervisé peut être vu comme un **problème d'optimisation** : parmi toutes les fonctions candidates  $f_\theta$ , nous cherchons celle dont les paramètres minimisent la fonction de coût.

La question devient alors : *comment trouver concrètement ces paramètres optimaux ?*

Dans certains cas, cette minimisation peut être résolue analytiquement, c'est-à-dire en obtenant une expression explicite des paramètres optimaux. Cependant, cette situation reste relativement rare et devient rapidement irréalisable lorsque les modèles gagnent en complexité.

On a alors recours à des méthodes d'optimisation qui cherchent progressivement à réduire la valeur de la fonction de coût. Parmi elles, l'une des plus importantes en Machine Learning est la *descente de gradient*, que nous présentons dans la section suivante.

### 2.1.3 Descente de gradient

L'idée de la descente de gradient est simple : imaginez que la fonction de coût représente un paysage de montagnes. Chaque configuration des paramètres  $\theta$  correspond à un point de ce paysage, tandis que l'altitude représente la valeur de la fonction de coût. L'objectif de l'apprentissage est alors de trouver le point le plus bas possible, c'est-à-dire les paramètres qui minimisent cette fonction.

#### Algorithme

À partir d'un point initial  $\theta^{(0)}$ , les paramètres sont mis à jour itérativement selon la règle :

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla \mathcal{C}(\theta^{(t)})$$

où :

- $\nabla \mathcal{C}(\theta)$  est le gradient de la fonction de coût ;
- $\eta > 0$  est le **learning rate**.

Le gradient indique la direction de plus forte augmentation de la fonction de coût. Afin de réduire cette dernière, l'algorithme se déplace donc dans la direction opposée au gradient.

À chaque itération, les paramètres sont ainsi légèrement modifiés afin de diminuer la valeur de la fonction de coût. Répétée un grand nombre de fois, cette procédure conduit généralement vers un minimum de la fonction.

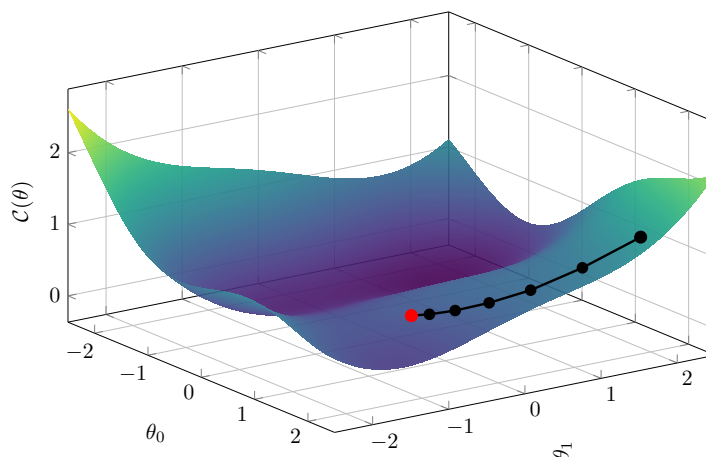


FIGURE 2.1 – Descente de gradient dans un espace à deux paramètres.

## Rôle du learning rate

Le paramètre  $\eta$  contrôle la taille des pas effectués lors de la descente. Son choix est crucial pour le bon fonctionnement de l'algorithme :

- si  $\eta$  est trop grand, les mises à jour peuvent dépasser le minimum recherché. L'algorithme risque alors d'osciller autour de celui-ci, voire de diverger complètement
- si  $\eta$  est trop petit, la fonction de coût diminue bien à chaque itération mais la convergence peut devenir extrêmement lente

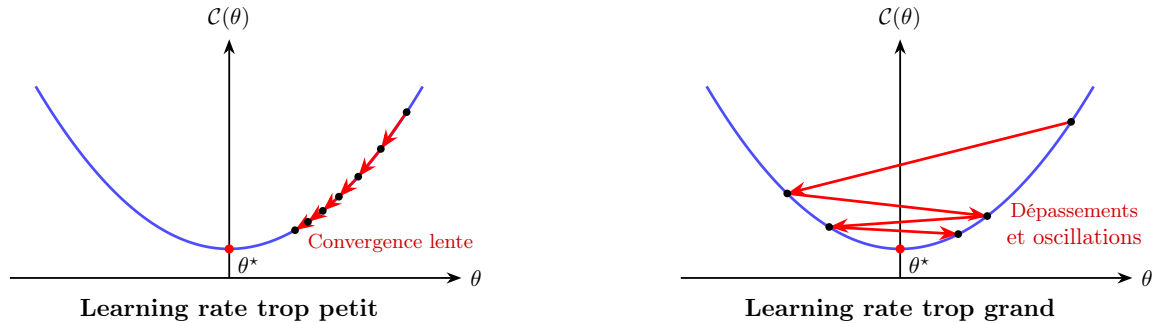


FIGURE 2.2 – Influence du learning rate  $\eta$  sur la descente de gradient.

Il n'existe pas de méthode universelle permettant de déterminer à l'avance le meilleur learning rate. En pratique, sa valeur est souvent choisie empiriquement. Par ailleurs, le learning rate n'est pas nécessairement constant au cours de l'apprentissage. Il est fréquent d'utiliser un taux d'apprentissage élevé au début afin de progresser rapidement vers une région intéressante de l'espace des paramètres, puis de le diminuer progressivement afin d'affiner la recherche du minimum.

## Cas des fonctions convexes

Le comportement de la descente de gradient dépend fortement de la forme de la fonction de coût.

Lorsque celle-ci est **convexe**, une propriété particulièrement intéressante apparaît : *tout minimum local est également un minimum global*.

Dans ce cas, si l'algorithme converge, il converge nécessairement vers la meilleure solution possible du problème d'optimisation.

Cette propriété explique pourquoi les fonctions de coût convexes occupent une place importante dans de nombreux modèles classiques de Machine Learning, notamment la régression linéaire que nous étudierons dans la section suivante.

## 2.2 Régression Linéaire Classique

Nous avons vu que l'apprentissage supervisé consiste à rechercher, parmi une famille de fonctions candidates, celle qui minimise une fonction de coût sur les données d'entraînement.

Une question reste alors : *quelle famille de fonctions choisir ?*

Le choix de cette famille dépend naturellement du problème étudié. Dans ce chapitre, nous nous intéressons aux problèmes de régression, c'est-à-dire aux situations où l'objectif est de prédire une quantité réelle : un prix, une température, une consommation électrique, etc..

L'une des familles de fonctions les plus simples et les plus utilisées est celle des **fonctions linéaires**. Cette hypothèse conduit au modèle de **régression linéaire**.

La régression linéaire repose sur une hypothèse simple : la variable à prédire peut être approximée par une *combinaison linéaire* des variables explicatives. Autrement dit, on suppose que les fonctions candidates sont de la forme :

$$f_{\theta}(x) = \theta_0 + \sum_{j=1}^d \theta_j x_j^{(i)} \quad (2.3)$$

où :

- $\theta_0$  est l'ordonnée à l'origine (*intercept*) ;
- $\theta_j$  mesure l'influence de la variable  $x_j$  sur la variable à prédire.

L'objectif est désormais de déterminer les coefficients

$$\theta_0, \dots, \theta_d$$

qui permettent au modèle de produire les meilleures prédictions possibles sur les données d'entraînement. Pour cela, il nous reste à définir la fonction de coût associée à la régression linéaire.

## Fonction de coût de la régression linéaire

Comme nous l'avons vu dans la formalisation de l'apprentissage supervisé, une fonction de perte mesure l'erreur commise sur une observation individuelle. Dans le cadre d'un problème de régression, plusieurs choix sont possibles. Par exemple, on pourrait mesurer l'erreur absolue :

$$\mathcal{L}(y, \hat{y}) = |y - \hat{y}|$$

ou encore l'erreur quadratique :

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$$

Ces deux fonctions pénalisent les erreurs de manière différente. En pratique, la perte quadratique est très largement utilisée pour plusieurs raisons :

- Elle accorde davantage d'importance aux grandes erreurs ;
- Elle est différentiable, ce qui permet d'utiliser efficacement des algorithmes d'optimisation tels que la descente de gradient ;
- Elle conduit à une fonction de coût convexe dans le cas de la régression linéaire, garantissant l'absence de minimum local sous-optimal ;

- Elle possède une solution analytique dans le cas de la régression linéaire, contrairement à de nombreux autres modèles de Machine Learning.

C'est donc la perte quadratique qui est retenue dans le cadre de la régression linéaire. La fonction de coût associée est donc :

$$\mathcal{C}(\theta) = \frac{1}{n} \sum_{i=1}^n \left( y^{(i)} - f_{\theta}(x^{(i)}) \right)^2 \quad (2.4)$$

En remplaçant l'expression du modèle donnée par l'équation (2.3), on obtient

$$\mathcal{C}(\theta) = \frac{1}{n} \sum_{i=1}^n \left( y^{(i)} - \theta_0 - \sum_{j=1}^d \theta_j x_j^{(i)} \right)^2 \quad (2.5)$$

L'apprentissage de la régression linéaire consiste alors à résoudre le problème d'optimisation

$$\theta^* = \arg \min_{\theta} \mathcal{C}(\theta) \quad (2.6)$$

Nous retrouvons ainsi exactement le cadre général présenté au début du chapitre : il s'agit de rechercher, parmi toutes les fonctions linéaires possibles, celle qui minimise la fonction de coût sur les données d'entraînement.

La question devient alors : *comment résoudre concrètement le problème d'optimisation (2.6) ?*

Deux approches sont possibles. La première consiste à utiliser un algorithme d'optimisation itératif, comme la *descente de gradient*, présenté précédemment. Cette approche est générale et s'applique à un très grand nombre de modèles d'apprentissage.

Cependant, la régression linéaire possède une propriété remarquable : la fonction de coût quadratique (2.5) permet dans certains cas, selon la nature des données, de calculer analytiquement le minimum.

### Résolution analytique : les équations normales

Pour résoudre analytiquement le problème d'optimisation, commençons par écrire sous forme matricielle notre modèle. Nous ajoutons une première colonne de 1 à la matrice des données afin de prendre en compte l'intercept  $\theta_0$ , on définit ainsi :

$$X = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_d^{(n)} \end{pmatrix}, \quad \theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{pmatrix}, \quad y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{pmatrix}$$

Les prédictions du modèle s'écrivent alors simplement

$$\hat{y} = X\theta$$

La fonction de coût peut ainsi être réécrite sous la forme :

$$\mathcal{C}(\theta) = \frac{1}{n}(y - X\theta)^T(y - X\theta)$$

Cette fonction étant différentiable, les paramètres optimaux sont obtenus en annulant son gradient. On obtient alors le système d'équations suivant, appelé **équations normales** :

$$X^T X \theta = X^T y$$

Si la matrice  $X^T X$  est inversible, on obtient directement la solution analytique

$$\theta^* = (X^T X)^{-1} X^T y \quad (2.7)$$

Cette solution n'existe que si la matrice  $X^T X$  est inversible, ce qui est équivalent à dire que la matrice  $X$  est de **rang plein**. Autrement dit, les variables explicatives doivent apporter des informations indépendantes : aucune colonne de  $X$  ne doit pouvoir s'écrire comme une combinaison linéaire des autres. Lorsque cette condition n'est pas vérifiée, plusieurs jeux de paramètres peuvent produire exactement les mêmes prédictions. La solution n'est alors plus unique et l'équation (2.7) ne peut pas être utilisée.

Cette situation apparaît notamment lorsque certaines variables sont fortement corrélées entre elles ou lorsque le nombre de variables est supérieur au nombre d'observations. Nous verrons dans la dernière partie de ce chapitre que les méthodes de régularisation, en particulier la régression Ridge, permettent précisément de résoudre ce problème.

## 2.3 Évaluer la performance d'une régression

Une fois le modèle appris avec la minimisation de la fonction de coût, il est nécessaire de mesurer sa capacité de généralisation sur un jeu de données indépendant. Pour cela, plusieurs métriques existent, chacune mettant en évidence un aspect particulier de la qualité des prédictions. Dans cette section, nous présentons les trois métriques les plus couramment utilisées pour évaluer un modèle de régression : la *Mean Squared Error* (MSE), la *Root Mean Squared Error* (RMSE) et le coefficient de détermination  $R^2$ .

### Mean Squared Error (MSE)

Pour chaque observation, le modèle commet une erreur appelée *résidu*, définie par

$$\epsilon^{(i)} = y^{(i)} - \hat{y}^{(i)}$$

Une manière naturelle d'évaluer un modèle consiste donc à mesurer l'importance moyenne de ces erreurs. Comme lors de l'apprentissage de la régression linéaire, on choisit de travailler avec les erreurs au carré. On obtient ainsi la **Mean Squared Error (MSE)** :

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 \quad (2.8)$$

La MSE est toujours positive et est nulle uniquement lorsque toutes les prédictions sont exactes. Plus sa valeur est faible, meilleur est le modèle.

L'utilisation du carré présente deux avantages importants :

- les erreurs positives et négatives ne s'annulent plus ;
- les erreurs importantes sont davantage pénalisées que les petites erreurs.

On remarque que cette métrique correspond exactement à la fonction de coût utilisée pour entraîner la régression linéaire. Il est donc naturel de l'utiliser également pour évaluer le modèle sur un jeu de données indépendant.

Son principal inconvénient est son manque d'interprétabilité : si la variable prédite est exprimée en euros, la MSE est exprimé en euros<sup>2</sup>, ce qui rend son interprétation peu intuitive.

## Root Mean Squared Error (RMSE)

Afin d'obtenir une métrique plus interprétable, on définit la **Root Mean Squared Error (RMSE)** comme la racine carrée de la MSE :

$$RMSE = \sqrt{MSE} \quad (2.9)$$

La RMSE conserve les propriétés de la MSE, notamment le fait de pénaliser fortement les erreurs importantes, mais elle possède un avantage majeur : elle s'exprime dans la même unité que la variable prédite. Par exemple, si l'on cherche à prédire le prix d'un logement exprimé en euros et que la RMSE vaut 2 500, cela signifie que les prédictions s'écartent en moyenne d'environ 2 500 euros des valeurs réelles.

Cette propriété rend la RMSE beaucoup plus facile à interpréter que la MSE, ce qui explique pourquoi elle est très souvent utilisée pour communiquer les performances d'un modèle.

## Coefficient de détermination $R^2$

Les deux métriques précédentes mesurent directement l'erreur de prédiction. Une autre approche consiste à comparer le modèle obtenu à une stratégie très naïve : prédire systématiquement la moyenne des observations.

Notons

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y^{(i)}$$

la moyenne des valeurs observées.

Le coefficient de détermination  $R^2$  compare alors l'erreur du modèle à celle obtenue en prédisant systématiquement cette moyenne :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (y^{(i)} - \bar{y})^2} \quad (2.10)$$

Le coefficient  $R^2$  possède une interprétation particulièrement intuitive :

- $R^2 = 1$  : les prédictions sont parfaites ;
- $R^2 = 0$  : le modèle ne fait pas mieux que de prédire systématiquement la moyenne ;
- $R^2 < 0$  : le modèle est moins performant que cette stratégie naïve.

Contrairement à la MSE et RMSE, le coefficient  $R^2$  est sans unité. Il fournit une mesure globale de la qualité du modèle, ce qui en fait l'une des métriques les plus utilisées pour évaluer la qualité globale d'une régression.

Aucune métrique n'est universellement meilleure qu'une autre. Leur choix dépend du contexte applicatif et des conséquences associées aux erreurs de prédiction.

## 2.4 Complexité des modèles

Nous disposons désormais d'un premier modèle de régression ainsi que des outils permettant d'évaluer ses performances. Une question naturelle se pose alors : *comment améliorer ces performances ?*

Une première approche consiste à augmenter la complexité du modèle afin de lui permettre de représenter des relations plus riches entre les variables explicatives et la variable à prédire. Cette section introduit cette notion de complexité à travers l'exemple des régressions polynomiales.

### 2.4.1 Régressions polynomiales

Dans les sections précédentes, nous avons présenté la régression linéaire comme un modèle permettant de prédire une variable quantitative à partir d'une combinaison linéaire des variables explicatives.

Une question naturelle se pose alors : *pourquoi se limiter à un modèle linéaire ?*

En effet, la relation entre la variable à prédire et les variables explicatives n'est pas toujours affine. Considérons par exemple le problème de la prédiction de la consommation de carburant d'un véhicule en fonction de sa vitesse.

À faible vitesse, la consommation est généralement élevée car le moteur fonctionne dans un régime peu efficace. Elle diminue ensuite lorsque la vitesse augmente, avant de réaugmenter à grande vitesse sous l'effet de la résistance de l'air. La relation entre la vitesse et la consommation est donc naturellement non linéaire.

Supposons que  $y$  représente la consommation de carburant et  $x$  la vitesse du véhicule. La régression linéaire suppose que cette relation est de la forme

$$f_{\theta}(x) = \theta_0 + \theta_1 x \quad (2.11)$$

Cette hypothèse peut être trop restrictive et ne permettre de capturer que la tendance générale des données. Une manière naturelle d'enrichir le modèle consiste à ajouter des puissances de la variable explicative. On obtient ainsi une **régression polynomiale**.

Par exemple, une régression polynomiale de degré 2 est définie par

$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 \quad (2.12)$$

Dans notre exemple, un tel modèle est capable de représenter la courbe de la relation entre la vitesse et la consommation de manière beaucoup plus fidèle qu'une simple droite.

Plus généralement, rien n'empêche d'augmenter le degré du polynôme afin de construire une famille de modèles de plus en plus flexible. Une régression polynomiale de degré  $p$  s'écrit

$$f_{\theta}(x) = \sum_{k=0}^p \theta_k x^k \quad (2.13)$$

Il est important de remarquer qu'une régression polynomiale n'est pas un nouvel algorithme d'apprentissage. Le modèle reste toujours linéaire par rapport aux paramètres  $\theta$ . La seule différence réside dans le choix des variables explicatives : au lieu d'utiliser uniquement les variables d'origine, on enrichit l'espace des variables en ajoutant leurs puissances et, dans des cas plus complexes, leurs interactions. Une fois cette transformation effectuée, la procédure d'apprentissage reste exactement la même que celle présentée pour la régression linéaire classique.

Voyons des exemples de surfaces que l'on se permet d'apprendre avec une telle méthode :

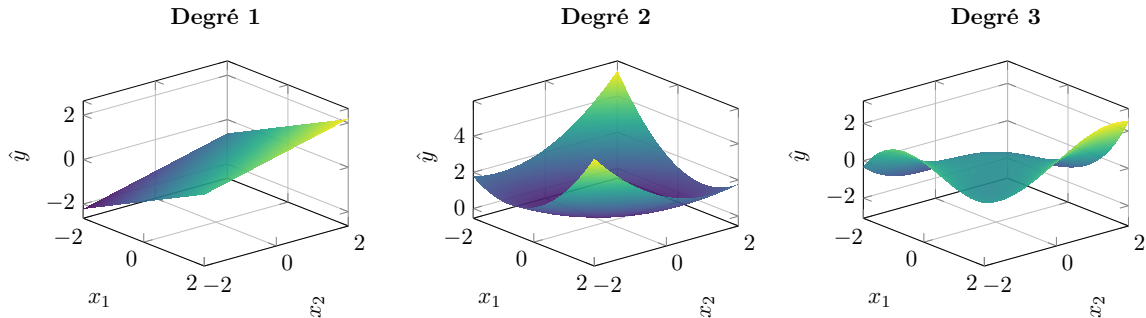


FIGURE 2.3 – Surfaces pouvant être apprises par une régression polynomiale avec deux variables explicatives.

On constate que l'augmentation du degré permet de représenter des relations de plus en plus complexes entre les variables. Cette idée repose sur un résultat fondamental de l'analyse mathématique.

**Théorème 1** (Théorème d'approximation de Weierstrass). *Soit  $f$  une fonction continue définie sur un intervalle fermé  $[a, b]$ . Alors, il existe une suite de polynômes qui converge uniformément vers  $f(x)$  pour  $x \in [a, b]$ .*

*Autrement dit, toute fonction continue par morceaux peut être arbitrairement bien approchée par une suite de polynômes.*

Ce résultat montre que la famille des polynômes est extrêmement riche : avec un degré suffisamment élevé, il est possible d'approcher une très grande variété de fonctions. Les

régressions polynomiales constituent ainsi une extension naturelle de la régression linéaire lorsque les relations entre les variables sont non linéaires.

Il serait alors tentant d'utiliser systématiquement des polynômes de degré très élevé afin d'obtenir le meilleur ajustement possible aux données d'entraînement. Cependant, cette stratégie présente plusieurs inconvénients :

- **Une variance plus importante** : plus le modèle est complexe, plus il devient sensible aux fluctuations propres aux données d'entraînement. De faibles variations dans les observations peuvent alors conduire à des modèles très différents ;
- **Un risque accru de surapprentissage** : un modèle trop flexible peut finir par apprendre le bruit présent dans les données plutôt que la véritable relation entre les variables. Bien que ses performances sur les données d'entraînement soient excellentes, sa capacité de généralisation sera alors fortement dégradée ;
- **une augmentation du coût de calcul** : Le nombre de variables explicatives croît rapidement avec le degré du polynôme, et plus encore lorsque plusieurs variables sont présentes. Cette augmentation de la dimension entraîne des coûts de calcul plus importants et peut rendre l'apprentissage plus difficile.

Ces observations mettent en évidence une question fondamentale en apprentissage automatique : *comment choisir la complexité d'un modèle afin d'obtenir les meilleures performances sur de nouvelles données ?*

La réponse repose sur un compromis entre deux sources d'erreur antagonistes : le biais et la variance, que nous présentons dans la section suivante.

## 2.4.2 Compromis Biais-Variance

Nous venons de voir qu'il est toujours possible d'augmenter la complexité d'un modèle en enrichissant la famille des fonctions qu'il est capable de représenter. Une question naturelle se pose alors : *pourquoi ne pas systématiquement choisir le modèle le plus complexe ?*

Pour répondre à cette question, il est utile de distinguer les différentes sources d'erreur pouvant affecter un modèle d'apprentissage.

**L'erreur d'approximation** correspond à l'incapacité de la famille de modèles choisie à représenter fidèlement la relation entre les variables. Un modèle trop simple (par exemple une droite pour représenter une relation quadratique) présente une erreur d'approximation élevée. Cette erreur diminue généralement lorsque la complexité du modèle augmente.

**L'erreur d'estimation** provient du fait que le modèle est appris à partir d'un nombre fini d'observations. Plus le modèle est complexe, plus il devient sensible aux fluctuations propres au jeu de données d'entraînement. Deux jeux de données légèrement différents peuvent alors conduire à des modèles très différents. Cette erreur tend donc à augmenter avec la complexité du modèle.

**L'erreur irréductible** correspond au bruit présent dans les données. Certaines informations influençant la variable à prédire ne sont tout simplement pas observées ou sont

intrinsèquement aléatoires. Cette erreur ne peut être supprimée, quel que soit le modèle utilisé.

La théorie de l'apprentissage statistique montre que l'erreur quadratique moyenne peut être décomposée en trois composantes :

$$\underbrace{\mathbb{E}[(Y - f_{\theta^*}(X))^2]}_{\text{MSE}} = (f(X) - \mathbb{E}[f_{\theta^*}(X)])^2 + \mathbb{E}[(f_{\theta^*}(X) - \mathbb{E}[f_{\theta^*}(X)])^2] + \sigma^2$$

$$= \underbrace{\text{Erreur d'approximation}}_{\text{Biais}^2} + \underbrace{\text{Erreur d'estimation}}_{\text{Variance}} + \text{Erreur irréductible}$$
(2.14)

Cette écriture met en évidence un phénomène fondamental : augmenter la richesse de l'espace des hypothèses permet généralement de réduire l'erreur d'approximation, puisqu'il devient plus probable que la véritable fonction appartienne à cet espace. En contrepartie, les paramètres du modèle deviennent plus sensibles aux fluctuations des données d'entraînement, ce qui augmente l'erreur d'estimation. Ce phénomène est connu sous le nom de **compromis biais-variance** illustré par la figure 2.4.

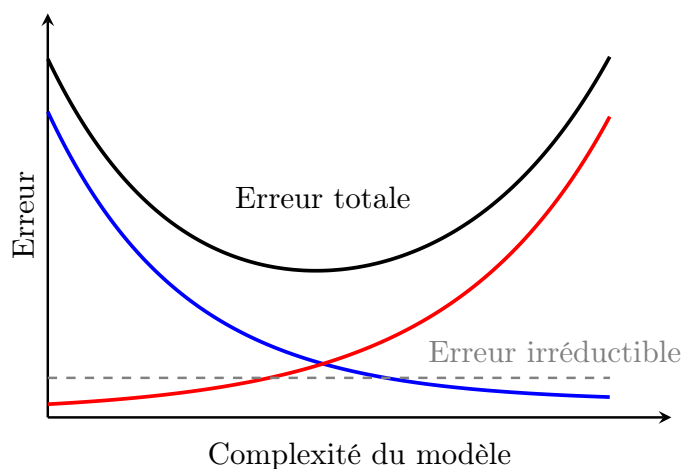


FIGURE 2.4 – Compromis **Biais**–**Variance**.

Ce constat rejoint une idée bien plus ancienne, celle du **rasoir d'Occam**. Un fondement philosophique attribué à Guillaume d'Occam (XIV<sup>e</sup> siècle), énonçant que, parmi plusieurs explications compatibles avec les observations, il convient de privilégier la plus simple.

En apprentissage automatique, cela peut être formulé de deux manières [3] :

1. Si deux modèles obtiennent les mêmes performances de généralisation, le modèle le plus simple est préférable, la simplicité étant une propriété souhaitable en elle-même.
2. Si deux modèles obtiennent les mêmes performances sur les données d'entraînement, le modèle le plus simple est généralement préférable, car il a davantage de chances de mieux généraliser à de nouvelles données.

La première formulation exprime un principe de parcimonie : lorsqu'aucun gain de performance n'est obtenu, il n'est pas nécessaire de retenir un modèle plus complexe. La seconde

traduit cette idée dans le contexte de l'apprentissage automatique en suggérant qu'un modèle moins complexe est souvent plus robuste face à de nouvelles données.

Le rasoir d'Occam reste aujourd'hui davantage un principe heuristique qu'un résultat universel. En particulier, Pedro Domingos [3] montre que la seconde formulation ne constitue pas une règle générale et que la première est elle-même sujette à discussion. Autrement dit, il n'existe pas de consensus théorique établissant que le modèle le plus simple est systématiquement le meilleur.

Néanmoins, en pratique, il est généralement recommandé de privilégier les modèles les plus simples capables d'atteindre les performances recherchées. Au-delà de leur capacité de généralisation souvent satisfaisante, ces modèles présentent plusieurs avantages : ils sont plus rapides à entraîner, plus faciles à interpréter, plus simples à maintenir et souvent moins sensibles aux fluctuations des données.

Cette philosophie s'accorde naturellement avec le compromis biais-variance présenté précédemment et conduit à introduire des mécanismes permettant de contrôler la complexité des modèles lors de leur apprentissage : c'est le principe de la **régularisation**.

### 2.4.3 Régressions pénalisées

Dans la section précédente, nous avons montré que des modèles trop complexes peuvent conduire à une mauvaise capacité de généralisation. Cette complexité ne provient cependant pas uniquement du choix d'un modèle très flexible, comme une régression polynomiale de degré élevé. Elle peut également être imposée par les données elles-mêmes.

Une première difficulté apparaît lorsque le nombre de variables explicatives devient très important par rapport au nombre d'observations. Le modèle doit alors estimer un grand nombre de paramètres, ce qui le rend plus sensible aux fluctuations des données d'entraînement. Cette situation est fréquente dans des domaines tels que la génétique, où plusieurs dizaines de milliers de gènes sont mesurés pour quelques centaines de patients, ou encore en imagerie, où une seule image peut être représentée par plusieurs milliers, voire millions, de variables.

Une seconde difficulté survient lorsque plusieurs variables explicatives sont fortement corrélées entre elles. Elles apportent alors une information redondante, ce qui rend l'estimation des paramètres plus instable.

D'un point de vue mathématique, ces difficultés apparaissent lorsque la matrice des variables explicatives  $X$  n'est pas de rang plein. Dans cette situation, plusieurs jeux de paramètres peuvent conduire à des performances très proches sur les données d'entraînement, tout en produisant des prédictions sensiblement différentes sur de nouvelles observations. Les coefficients estimés deviennent alors très sensibles aux fluctuations des données, ce qui favorise le sur-apprentissage.

Nous avons vu que la régression linéaire classique consiste à résoudre le problème d'optimisation (2.6)

$$\theta^* = \arg \min_{\theta} \mathcal{C}(\theta)$$

Une idée naturelle consiste alors à ne plus chercher uniquement le modèle minimisant la fonction de coût, mais également à limiter sa complexité. C'est précisément le principe de la **régularisation**, qui consiste à modifier le problème d'optimisation en ajoutant un terme pénalisant les modèles jugés trop complexes.

On définit ainsi une nouvelle fonction de coût

$$\mathcal{C}_{\text{reg}}(\theta) = \mathcal{C}(\theta) + \lambda \Omega(\theta) \quad (2.15)$$

où :

- $\mathcal{C}(\theta)$  mesure la qualité de l'ajustement aux données ;
- $\Omega(\theta)$  est une fonction de pénalisation mesurant la complexité du modèle, généralement à partir de la taille de ses coefficients ;
- $\lambda \geq 0$  est un hyperparamètre contrôlant le compromis entre qualité d'ajustement et complexité.

Le problème d'optimisation devient alors

$$\theta^* = \arg \min_{\theta} \mathcal{C}_{\text{reg}}(\theta) \quad (2.16)$$

Lorsque  $\lambda = 0$ , aucune pénalisation n'est appliquée et l'on retrouve la régression linéaire classique. À mesure que  $\lambda$  augmente, les modèles les plus complexes sont davantage pénalisés. Le choix de la fonction de pénalisation  $\Omega$  conduit à différentes méthodes de régularisation. Les trois plus utilisées sont les régressions **Ridge**, **Lasso** et **ElasticNet**, que nous présentons dans les sections suivantes.

## Régression Ridge

Le principe de la régression **Ridge** consiste à pénaliser les modèles dont les coefficients sont de grande amplitude pour limiter leur complexité. L'idée est qu'un modèle reposant sur des coefficients très élevés est généralement plus sensible aux fluctuations des données qu'un modèle dont les coefficients restent modérés.

Pour cela, la fonction de coût des moindres carrés est complétée par un terme de pénalisation proportionnel à la norme euclidienne des paramètres, appelée pénalité  $L_2$ . Le problème d'optimisation devient alors

$$\theta^* = \arg \min_{\theta} (\|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2) \quad (2.17)$$

où

- $\|y - X\theta\|_2^2$  mesure l'erreur d'ajustement du modèle ;
- $\|\theta\|_2^2 = \sum_{j=1}^d \theta_j^2$  pénalise les coefficients de grande amplitude ;
- $\lambda \geq 0$  contrôle l'importance de cette pénalisation.

À mesure que  $\lambda$  augmente, les coefficients sont progressivement rapprochés de zéro. Le modèle devient alors plus stable, moins sensible aux fluctuations des données d'entraînement et présente généralement une meilleure capacité de généralisation. En revanche, une valeur trop importante de  $\lambda$  peut conduire à un modèle trop contraint et donc à un phénomène de sous-apprentissage (*underfitting*).

Comme dans le cas de la régression linéaire classique, le problème d'optimisation de la régression Ridge est convexe et différentiable. Il admet donc une solution analytique, donnée par

$$\theta^* = \left( X^\top X + \lambda I \right)^{-1} X^\top y \quad (2.18)$$

avec  $I$  la matrice identité.

Cette expression est très proche de celle obtenue pour la régression linéaire classique (2.7). La seule différence réside dans l'ajout du terme  $\lambda I$  à la matrice  $X^\top X$ . Cette modification possède une propriété fondamentale : pour tout  $\lambda > 0$ , la matrice  $X^\top X + \lambda I$  est inversible, même lorsque  $X^\top X$  ne l'est pas. La régression Ridge admet ainsi toujours une solution unique.

## Régression Lasso

La régression Ridge permet de réduire la complexité d'un modèle en pénalisant les coefficients de grande amplitude. Cependant, elle présente une limite importante : les coefficients sont uniquement rapprochés de zéro sans jamais devenir exactement nuls. Toutes les variables explicatives continuent donc à intervenir dans le modèle.

Or, dans de nombreux problèmes, seule une faible partie des variables est réellement utile à la prédiction. C'est notamment le cas en génétique, où plusieurs dizaines de milliers de gènes peuvent être mesurés alors que seuls quelques-uns sont impliqués dans le développement d'une maladie. Dans une telle situation, il est souhaitable de construire un modèle capable d'identifier automatiquement les variables les plus pertinentes et d'écarter celles qui n'apportent aucune information.

La régression **Lasso** (*Least Absolute Shrinkage and Selection Operator*) répond précisément à cet objectif en remplaçant la pénalisation fondée sur la norme  $L_2$  par une pénalisation utilisant la norme  $L_1$ . Le problème d'optimisation devient alors

$$\theta^* = \arg \min_{\theta} (\|y - X\theta\|_2^2 + \lambda \|\theta\|_1) \quad (2.19)$$

où

$$\|\theta\|_1 = \sum_{j=1}^d |\theta_j|$$

Comme pour la régression Ridge, le paramètre  $\lambda$  contrôle l'intensité de la régularisation. Plus  $\lambda$  augmente, plus les coefficients sont pénalisés.

La différence essentielle entre les pénalisations  $L_1$  et  $L_2$  réside dans leur comportement au voisinage de zéro. La pénalisation  $L_2$  diminue très rapidement lorsque les coefficients deviennent petits. À l'inverse, la pénalisation  $L_1$  continue à pénaliser les petits coefficients de manière proportionnelle à leur valeur absolue. L'algorithme d'optimisation a ainsi tendance à annuler complètement les coefficients dont la contribution à la prédiction est faible, plutôt qu'à simplement les réduire.

Cette propriété fait de la régression Lasso une méthode de **sélection automatique de variables**. Les variables dont le coefficient devient nul sont retirées du modèle, tandis que seules les variables les plus pertinentes sont conservées.

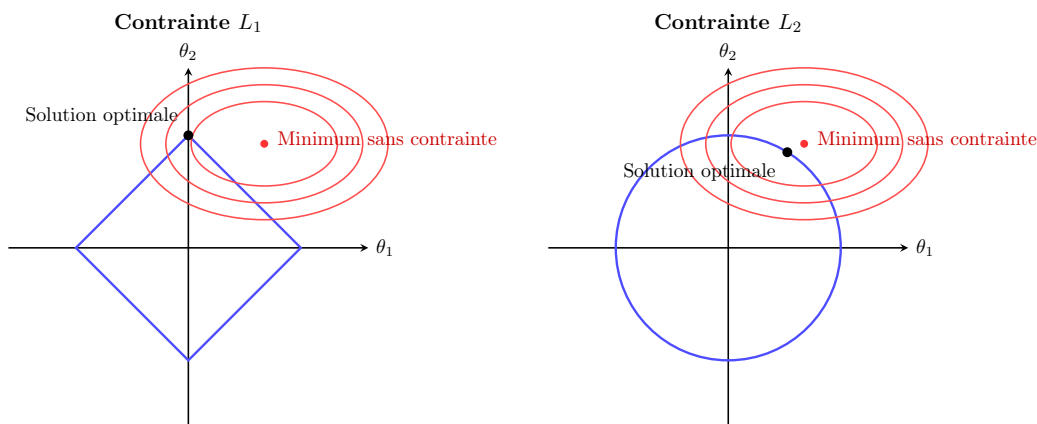


FIGURE 2.5 – Comparaison géométrique des contraintes  $L_1$  et  $L_2$ .

Contrairement à la régression Ridge, la régression Lasso ne possède pas de solution analytique simple. En effet, la présence de la valeur absolue dans la norme  $L_1$  rend la fonction de coût non différentiable lorsque l'un des coefficients est nul. Les paramètres sont donc estimés à l'aide d'algorithmes d'optimisation numérique, tels que la descente de coordonnées (*Coordinate Descent*) ou les méthodes de gradient proximal.

## Régression ElasticNet

La régression Ridge est particulièrement adaptée lorsque la plupart des variables explicatives apportent de l'information et sont fortement corrélées. À l'inverse, la régression Lasso est préférable lorsque seules quelques variables sont réellement pertinentes, puisqu'elle réalise automatiquement une sélection de variables.

En pratique, ces deux situations coexistent fréquemment. Certaines variables sont redondantes, tandis que d'autres sont peu informatives. La régression **Elastic Net** a été proposée afin de combiner les avantages des approches Ridge et Lasso.

Le problème d'optimisation devient alors

$$\theta^* = \arg \min_{\theta} (\|y - X\theta\|_2^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2) \quad (2.20)$$

où les paramètres  $\lambda_1$  et  $\lambda_2$  contrôlent respectivement les pénalisations  $L_1$  et  $L_2$ .

La pénalisation  $L_2$  permet de stabiliser l'estimation des coefficients en présence de variables fortement corrélées, tandis que la pénalisation  $L_1$  réalise une sélection automatique des variables les moins informatives. Elastic Net constitue ainsi un compromis entre les approches Ridge et Lasso.

Comme pour la régression Lasso, ce problème ne possède pas de solution analytique simple et est résolu à l'aide d'algorithmes d'optimisation numérique.

Ainsi, les méthodes de régularisation poursuivent toutes le même objectif : contrôler la complexité des modèles afin d'améliorer leur capacité de généralisation. En pénalisant les coefficients, elles réduisent leur sensibilité aux fluctuations des données d'entraînement. Cette diminution de la complexité se traduit généralement par une baisse de la variance du modèle, au prix d'une légère augmentation de son biais. Le paramètre de régularisation contrôle ainsi directement le compromis **biais-variance** présenté précédemment.

# Bibliographie

- [1] Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of k-fold cross-validation. In Sebastian Thrun, Lawrence K. Saul, and Bernhard Schölkopf, editors, *Advances in Neural Information Processing Systems 16 (NeurIPS 2003)*, pages 513–520. MIT Press, 2003.
- [2] Sylvain Arlot and Alain Celisse. A survey of cross-validation procedures for model selection. *Statistics Surveys*, pages 40–79, 2010.
- [3] Pedro Domingos. The role of occam’s razor in knowledge discovery. *Data Mining and Knowledge Discovery*, pages 409–425, 1999.